

Strategy Pattern

Polimorfismo por Composição

Por que o nome Strategy?

A palavra *Strategy* significa **estratégia**: uma maneira escolhida para realizar uma tarefa.

Pense em uma viagem

Para chegar ao mesmo destino, podemos escolher:

- carro;
- ônibus;
- avião.

O destino é o mesmo, mas a maneira de chegar até ele pode mudar.

No código

O padrão Strategy permite definir diferentes maneiras de executar uma tarefa e escolher qual delas será utilizada.

Em resumo

Strategy permite trocar a maneira de executar uma tarefa.

Dois exemplos, um mesmo padrão

Vamos ver o padrão **Strategy** aplicado em dois contextos diferentes:

- retomando o exemplo de **formas de pagamento** (Pagavel), quando o comportamento precisa **poder ser trocado** antes de finalizar um pedido;
- um exemplo novo, de **evolução de jogador** em um jogo, mostrando por que herança falha quando uma característica muda **durante a vida do objeto**.

Fio condutor

Em ambos, o mesmo problema aparece: como trocar **o que um objeto faz**, sem trocar **o objeto em si**?

E se o comportamento precisar mudar em tempo de execução?

Até agora, escolhemos a forma de pagamento **na hora de criar o objeto**.

Novo cenário

E se um Pedido precisar começar com uma forma de pagamento, mas o cliente puder **trocar de ideia** antes de finalizar a compra?

Precisamos de um objeto que:

- guarde uma **referência** para uma forma de pagamento (do tipo `Pagavel`);
- permita **trocar** essa referência a qualquer momento, antes de finalizar o pedido.

Construindo o Pedido

```
public class Pedido {  
    private double valorTotal;  
    private Pagavel formaPagamento;  
  
    public Pedido(double valorTotal, Pagavel formaPagamento) {  
        this.valorTotal = valorTotal;  
        this.formaPagamento = formaPagamento;  
    }  
  
    public void setFormaPagamento(Pagavel formaPagamento) {  
        this.formaPagamento = formaPagamento;  
    }  
  
    public void finalizar() {  
        formaPagamento.pagar(valorTotal);  
    }  
}
```

Pedido não sabe (nem precisa saber) qual forma de pagamento será usada – ele só delega a tarefa para formaPagamento.

Trocando a estratégia em tempo de execução

```
Pedido pedido = new Pedido(150.0, new Boleto("00190.00009"));

// O cliente muda de ideia antes de finalizar:
pedido.setFormaPagamento(new Pix("cliente@email.com"));

pedido.finalizar();
// "Transferindo R$ 150.0 via Pix cliente@email.com"
```

O que aconteceu aqui

O mesmo objeto Pedido mudou de comportamento em tempo de execução, apenas trocando **qual implementação** de `Pagavel` ele usa internamente – sem nenhum `if` para verificar o tipo.

O padrão Strategy

Definição

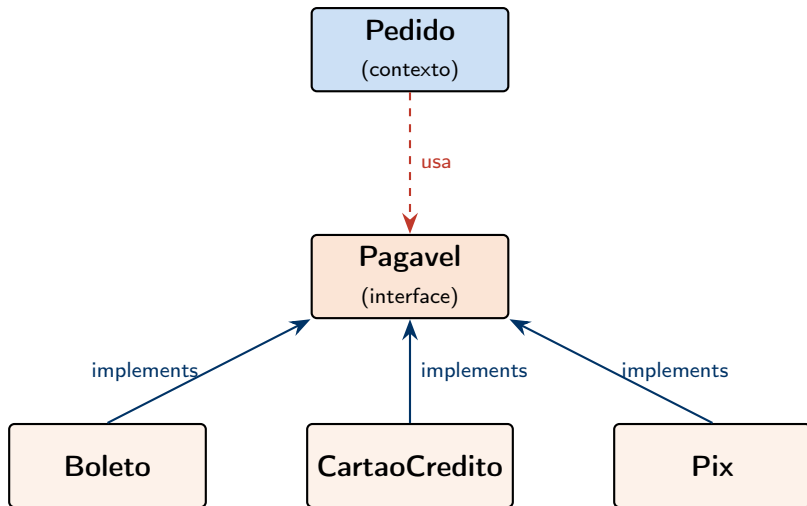
Strategy é um padrão de projeto em que um objeto (o **contexto**) delega uma tarefa a outro objeto, escolhido através de uma **interface comum** – e essa escolha pode ser trocada em tempo de execução.

No nosso exemplo:

- Pedido é o **contexto**;
- Pagavel é a **interface da estratégia**;
- Boleto, CartaoCredito e Pix são as **estratégias concretas**.

Vocês já construíram um Strategy sem saber o nome dele.

Diagrama do padrão Strategy



Pedido conhece apenas a interface Pagavel – nunca as classes concretas.

Strategy no dia a dia do Java: Comparator

Você já usou Strategy sem perceber, lá na interface List:

```
List<String> nomes = new ArrayList<>();
nomes.add("Rex");
nomes.add("Bidu");
nomes.add("Amy");

Collections.sort(nomes); // ordem alfabética padrão

Collections.sort(nomes, Comparator.comparing(String::length));
// agora ordena por tamanho do nome
```

A relação com Strategy

Comparator é uma interface: o contrato de "como comparar dois elementos". Collections.sort é o **contexto**, e cada Comparator diferente é uma **estratégia de ordenação** diferente.

Contexto: sistema de XP de um jogo

Um jogo calcula a experiencia (XP) ganha pelos jogadores ao final de cada partida.

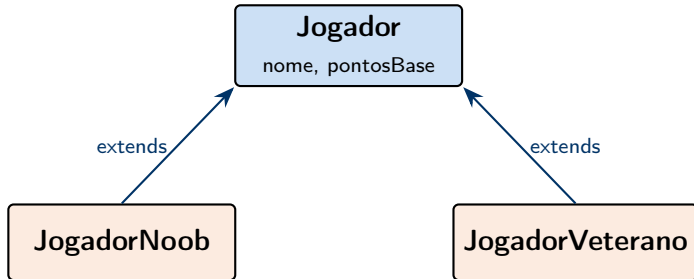
- jogadores **Noob** ganham XP direto, sem bonus;
- jogadores **Veterano** ganham XP multiplicado, mais um bonus fixo por experiencia;
- todo jogador possui nome e uma pontuacao base obtida na partida.

Uma decisao de projeto

Como Noob e Veterano calculam XP de formas diferentes, isso parece um caso classico de **heranca com sobrescrita de metodo**.

Modelando com heranca

- Jogador – classe abstrata, com nome, pontos base e o metodo abstrato calcularXp();
- JogadorNoob extends Jogador – XP igual a pontuacao base;
- JogadorVeterano extends Jogador – XP com multiplicador e bonus.



Codigo: classe abstrata Jogador

```
public abstract class Jogador {  
    private String nome;  
    private double pontosBase;  
  
    public Jogador(String nome, double pontosBase) {  
        this.nome = nome;  
        this.pontosBase = pontosBase;  
    }  
  
    public String getNome() { return nome; }  
    public double getPontosBase() { return pontosBase; }  
  
    public abstract double calcularXp();  
}
```

O metodo `calcularXp()` nao tem implementacao aqui – cada subclasse decide como calcular.

Codigo: JogadorNoob e JogadorVeterano

```
public class JogadorNoob extends Jogador {
    public JogadorNoob(String nome, double pontosBase) {
        super(nome, pontosBase);
    }

    @Override
    public double calcularXp() {
        return getPontosBase() * 1.0;
    }
}

public class JogadorVeterano extends Jogador {
    private double bonusVeterano;

    public JogadorVeterano(String nome, double pontosBase,
                           double bonusVeterano) {
        super(nome, pontosBase);
        this.bonusVeterano = bonusVeterano;
    }

    @Override
    public double calcularXp() {
        return (getPontosBase() * 2.5) + bonusVeterano;
    }
}
```

Usando o sistema

```
JogadorNoob ana = new JogadorNoob("Ana", 100);  
System.out.println(ana.calcularXp()); // 100.0
```

Ate aqui, tudo bem

A heranca funciona perfeitamente enquanto cada jogador **nasce e permanece** na mesma categoria durante todo o jogo.

Mas os jogos costumam ter progressao: um jogador pode **deixar de ser Noob e virar Veterano**.

O problema: Ana evolui de Noob para Veterana

A situacao

Ana acumulou pontuacao suficiente e o jogo decidiu promove-la de Noob para Veterana.

- a variavel `ana` foi declarada como `JogadorNoob`;
- o **tipo** de um objeto em Java **nao pode ser alterado depois que ele foi criado**;
- um objeto `JogadorNoob` sera sempre um `JogadorNoob` durante toda a sua vida.

Pergunta

Como fazer `ana` se tornar uma `JogadorVeterano`?

Tentando resolver com heranca

A unica saida e criar **um novo objeto** e copiar os dados manualmente:

```
JogadorNoob ana = new JogadorNoob("Ana", 100);  
  
// Ana evoluiu... precisamos criar um objeto novo:  
JogadorVeterano anaVeterana = new JogadorVeterano(  
    ana.getNome(), ana.getPontosBase(), 50.0);  
  
// A partir de agora, o resto do codigo do jogo precisa  
// trocar toda referencia a "ana" por "anaVeterana".
```

Observacao

O objeto original ana **continua existindo** como JogadorNoob em qualquer outro lugar do sistema que ainda aponte para ele.

Por que essa solucao e ruim?

- precisamos copiar **manualmente** cada atributo de um objeto para o outro – e se Jogador tivesse inventario, historico de partidas, conquistas, tudo isso precisaria ser copiado tambem;
- se esquecermos de copiar um atributo, temos um **bug silencioso** – o codigo compila normalmente;
- o objeto antigo continua vivo em qualquer variavel, lista ou colecao que ja apontava para ele – criando **duas versoes** da Ana no sistema;
- se o jogo tivesse mais categorias (Noob, Intermediario, Veterano, Lenda), teriamos que repetir esse processo de copia **a cada promocao**.

A raiz do problema

O que aconteceu

Modelamos a **categoria do jogador** (Noob ou Veterano) como sendo o **próprio tipo da classe**.

Mas, no mundo real do jogo, a categoria **nao e algo fixo** – ela e algo que **muda ao longo do tempo**, para o mesmo jogador.

Ideia central

Se algo pode mudar durante a vida do objeto, talvez ele **nao devesse ser o tipo da classe**, e sim **um atributo** que pode ser trocado.

Refatorando: a categoria vira uma interface

Em vez de JogadorNoob e JogadorVeterano serem **subclasses** de Jogador, elas viram **estrategias de calculo de XP**.

- Jogador passa a ser **uma unica classe**, sem subclasses de categoria;
- a categoria vira um **atributo** do tipo EstrategiaXp, uma interface;
- EstrategiaXpNoob e EstrategiaXpVeterano implementam essa interface, cada uma com sua formula.

Codigo: a interface EstrategiaXp

```
public interface EstrategiaXp {  
    double calcular(double pontosBase);  
}
```

O contrato

Toda estrategia de XP promete saber calcular um valor de XP a partir da pontuacao base – sem dizer, de antemao, qual e a formula usada.

Codigo: as estrategias concretas

```
public class EstrategiaXpNoob implements EstrategiaXp {
    @Override
    public double calcular(double pontosBase) {
        return pontosBase * 1.0;
    }
}

public class EstrategiaXpVeterano implements EstrategiaXp {
    private double bonus;

    public EstrategiaXpVeterano(double bonus) {
        this.bonus = bonus;
    }

    @Override
    public double calcular(double pontosBase) {
        return (pontosBase * 2.5) + bonus;
    }
}
```

A formula de cada categoria e exatamente a mesma de antes – so mudou **onde** ela mora.

Codigo: a nova classe Jogador (unica)

```
public class Jogador {
    private String nome;
    private double pontosBase;
    private EstrategiaXp estrategiaXp;

    public Jogador(String nome, double pontosBase,
                    EstrategiaXp estrategiaXp) {
        this.nome = nome;
        this.pontosBase = pontosBase;
        this.estrategiaXp = estrategiaXp;
    }

    public void setEstrategiaXp(EstrategiaXp estrategiaXp) {
        this.estrategiaXp = estrategiaXp;
    }

    public double calcularXp() {
        return estrategiaXp.calcular(pontosBase);
    }
}
```

Nao existe mais JogadorNoob nem JogadorVeterano – existe apenas Jogador, com uma estrategia trocavel

Usando a nova versao

```
Jogador ana = new Jogador("Ana", 100, new EstrategiaXpNoob());  
System.out.println(ana.calcularXp()); // 100.0
```

Ate aqui, o comportamento e identico ao da versao com heranca – a diferenca aparece na hora da promocao.

Ana evolui: apenas trocando a estrategia

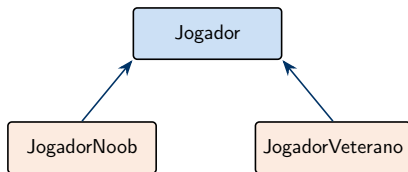
```
Jogador ana = new Jogador("Ana", 100, new EstrategiaXpNoob());  
System.out.println(ana.calcularXp()); // 100.0  
  
// Ana evoluiu e virou veterana:  
ana.setEstrategiaXp(new EstrategiaXpVeterano(50.0));  
System.out.println(ana.calcularXp()); // 300.0
```

A diferenca essencial

ana e o **mesmo objeto** do inicio ao fim. Nao criamos um objeto novo, nao copiamos nenhum atributo, e qualquer outra parte do sistema que ja tinha uma referencia para ana **ve a mudanca automaticamente**.

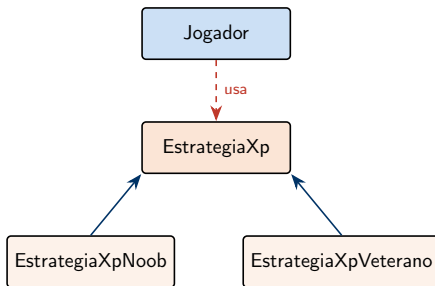
Heranca vs Strategy: antes e depois

ANTES (heranca)



trocar de categoria
= trocar de objeto

DEPOIS (Strategy)



trocar de categoria
= trocar a estrategia

Recapitulando

- heranca modela algo que **define permanentemente o que o objeto e** – um JogadorNoob nunca deixa de ser JogadorNoob;
- quando uma caracteristica pode **mudar durante a vida do objeto**, modela-la como tipo da classe cria um problema: para "mudar de tipo", somos obrigados a criar um objeto novo e copiar tudo manualmente;
- o Strategy resolve isso transformando esse comportamento variavel em **um atributo**, guardado atras de uma interface;
- o resultado e o **mesmo objeto**, com identidade preservada, apenas **trocando de comportamento** em tempo de execucao.

Strategy: o mesmo princípio em dois contextos

Papel	Pagamento	Jogador
Contexto	Pedido	Jogador
Interface da estratégia	Pagavel	EstrategiaXp
Estratégias concretas	Boleto, CartaoCredito, Pix	EstrategiaXpNoob, EstrategiaXpVeterano
Troca em tempo de execução	setFormaPagamento(.setEstrategiaXp(...))	

A mesma ideia, sempre

Um **contexto** guarda uma referência a uma **interface**. Trocar de comportamento em tempo de execução é só trocar **qual implementação** essa referência aponta – o objeto do contexto nunca muda de identidade.